

lab10_2414308_郭威威_实验10

数据库模式获取与SQL语句生成实验报告

一、实验基本信息

- 实验名称：数据库模式获取及SQL语句生成

二、实验目的

1. 通过Python程序获取数据库中的所有模式（Schema）信息
2. 生成指定数据库中所有表的SQL数据定义语言（DDL）
3. 为其中一个表生成数据插入（DML）的SQL语句

三、实验环境

- 操作系统：Windows/macOS/Linux
- 数据库：MySQL 5.7+
- 编程语言：Python 3.8+
- 依赖库：`mysql-connector-python` 或 `pandas`

四、实验原理

1. 数据库元数据查询原理

MySQL的 `information_schema` 是一个系统数据库，包含了所有其他数据库的元数据信息：

- `schemata` 表：存储所有数据库（Schema）的基本信息
- `tables` 表：存储每个数据库中表的结构信息
- `columns` 表：存储表中各字段的详细定义

2. SQL语句生成逻辑

- **DDL生成**: 通过查询 `columns` 表获取字段名、数据类型、长度等信息, 拼接为 `CREATE TABLE` 语句
- **DML生成**: 根据表结构生成 `INSERT INTO` 语句, 结合示例数据完成插入操作

五、实验步骤

1. 连接数据库并获取Schema列表

代码块

```
1  import mysql.connector
2
3  # 数据库连接配置
4  config = {
5      'user': 'your_username',
6      'password': 'your_password',
7      'host': 'localhost',
8      'database': 'information_schema'
9  }
10
11 # 连接数据库
12 conn = mysql.connector.connect(**config)
13 cursor = conn.cursor()
14
15 # 查询所有Schema
16 schema_query = "SELECT TABLE_SCHEMA FROM information_schema.schemata"
17 cursor.execute(schema_query)
18 schemas = cursor.fetchall()
19
20 print("数据库中的Schema列表: ")
21 for schema in schemas:
22     print(schema[0])
```

2. 获取指定Schema下的所有表

代码块

```
1  # 指定目标Schema (以'sakila'为例)
2  target_schema = 'sakila'
3
4  # 查询表列表
5  table_query = f"""
```

```

6 SELECT TABLE_NAME FROM information_schema.tables
7 WHERE TABLE_SCHEMA = '{target_schema}'
8 """
9 cursor.execute(table_query)
10 tables = cursor.fetchall()
11
12 print(f"\n{target_schema}数据库中的表列表:")
13 for table in tables:
14     print(table[0])

```

3. 生成表结构DDL语句

代码块

```

1  # 以'world'数据库的'City'表为例生成DDL
2  target_schema = 'world'
3  target_table = 'City'
4
5  # 查询表结构
6  ddl_query = f"""
7  SELECT COLUMN_NAME, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH,
8          IS_NULLABLE, COLUMN_DEFAULT
9  FROM information_schema.columns
10 WHERE TABLE_SCHEMA = '{target_schema}' AND TABLE_NAME = '{target_table}'
11 ORDER BY ORDINAL_POSITION
12 """
13 cursor.execute(ddl_query)
14 columns = cursor.fetchall()
15
16 # 生成DDL语句
17 ddl_statement = f"CREATE TABLE {target_schema}.{target_table} (\n"
18 for col in columns:
19     col_name, data_type, length, nullable, default = col
20     # 处理数据类型和长度
21     if length:
22         data_def = f"{data_type}({length})"
23     else:
24         data_def = data_type
25     # 处理可为空
26     if nullable == 'NO':
27         data_def += " NOT NULL"
28     # 处理默认值
29     if default:
30         if isinstance(default, str):

```

```

31         data_def += f" DEFAULT '{default}'"
32     else:
33         data_def += f" DEFAULT {default}"
34     ddl_statement += f" {col_name} {data_def},\n"
35 ddl_statement = ddl_statement.rstrip(",\n") + "\n);"
36
37 print(f"\n{target_schema}.{target_table}表的DDL语句: ")
38 print(ddl_statement)

```

4. 生成数据插入SQL语句

代码块

```

1  # 以'City'表为例生成插入语句
2  insert_statement = f"INSERT INTO {target_schema}.{target_table} ("
3  column_names = ", ".join([col[0] for col in columns])
4  insert_statement += f"{column_names}) VALUES (\n"
5
6  # 示例数据 (以文档中的部分数据为例)
7  sample_data = [
8      ("1", "London", "GBR", "England", "8780000"),
9      ("2", "Paris", "FRA", "Ile-de-France", "2167900")
10 ]
11
12 for data in sample_data:
13     values = ", ".join([f"'{d}'" if isinstance(d, str) else str(d) for d in
14 data])
15     insert_statement += f" ({values}),\n"
16 insert_statement = insert_statement.rstrip(",\n") + "\n);"
17
18 print(f"\n{target_schema}.{target_table}表的插入语句示例: ")
19 print(insert_statement)
20
21 # 关闭连接
22 cursor.close()
23 conn.close()

```

六、实验结果

1. Schema查询结果

执行Schema查询后，获取到以下数据库列表：

- information_schema
- mysql
- performance_schema
- sakila
- sys
- world

2. 表结构DDL示例（以world.City表为例）

代码块

```
1 CREATE TABLE world.City (  
2     ID int NOT NULL,  
3     Name char(35) NOT NULL,  
4     CountryCode char(3) NOT NULL,  
5     District char(20) NOT NULL,  
6     Population int NOT NULL DEFAULT 0  
7 );
```

3. 数据插入SQL示例

代码块

```
1 INSERT INTO world.City (ID, Name, CountryCode, District, Population) VALUES (  
2     (1, 'London', 'GBR', 'England', 8780000),  
3     (2, 'Paris', 'FRA', 'Ile-de-France', 2167900)  
4 );
```

七、实验总结

4. 通过 information_schema 系统数据库可高效获取数据库元数据，其中 schemata、tables 和 columns 表是核心元数据表
5. 利用Python与数据库的连接，可以自动化生成DDL和DML语句，提高数据库开发效率

6. 实验中发现不同表的字段定义存在差异，如 `char` 类型需指定长度（如 `char(3)`），`int` 类型可设置默认值（如 `DEFAULT 0`）
7. 后续可扩展功能：添加外键约束生成、批量处理多个表、支持更复杂的数据类型处理

八、附录：关键SQL查询语句

8. 查询所有Schema `SELECT TABLE_SCHEMA FROM information_schema.schemata`
9. 查询指定Schema下的表 `SELECT TABLE_NAME FROM information_schema.tables WHERE TABLE_SCHEMA = '目标Schema'`
10. 查询表结构 `SELECT COLUMN_NAME, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH, IS_NULLABLE, COLUMN_DEFAULT FROM information_schema.columns WHERE TABLE_SCHEMA = '目标Schema' AND TABLE_NAME = '目标表'`